

多智能体图表复现系统 - API 接口文档

版本: v3.1.2

更新日期: 2026年4月

基础URL: http://localhost:8001

目录

- 1. 接口概述
- 2. 认证说明
- 3. 通用响应格式
- 4. 状态码说明
- 5. 文件上传接口
- 6. 流水线管理接口
- 7. 结果查询接口
- 8. 文件下载接口
- 9. 历史记录接口
- 10. WebSocket 接口
- 11. 错误处理
- 12. 使用示例

1. 接口概述

1.1 API 基础信息

项目	说明
协议	HTTP/1.1, WebSocket
基础URL	http://localhost:8001
数据格式	JSON, multipart/form-data
字符编码	UTF-8
时区	UTC

1.2 接口分类

系统提供以下几类接口：

- **文件上传:** 上传参考图表图片
- **流水线管理:** 启动、停止、查询流水线状态
- **结果查询:** 获取流水线执行结果和详细报告
- **文件下载:** 下载生成的代码、图片和报告
- **历史记录:** 查询和管理历史执行记录
- **实时通信:** WebSocket 实时推送执行进度

1.3 接口列表

序号	接口路径	方法	功能描述
1	/health	GET	健康检查
2	/api/upload	POST	上传图片
3	/api/pipeline/start	POST	启动流水线
4	/api/pipeline/{id}/status	GET	查询流水线状态
5	/api/pipeline/{id}/stop	POST	停止流水线
6	/api/results/{id}	GET	获取执行结果
7	/api/download/{id}/code	GET	下载代码文件
8	/api/download/{id}/image	GET	下载图片文件
9	/api/download/{id}/report	GET	下载报告文件
10	/api/history	GET	获取历史记录
11	/api/history/{id}	DELETE	删除历史记录
12	/ws/{id}	WebSocket	实时通信

2. 认证说明

2.1 当前版本

当前版本（v1.0.0）暂不需要认证，所有接口均可直接访问。

2.2 未来版本

后续版本可能会引入以下认证机制：

- Bearer Token 认证
- API Key 认证
- OAuth 2.0 认证

3. 通用响应格式

3.1 成功响应

```
{
  "field1": "value1",
  "field2": "value2"
}
```

3.2 错误响应

```
{
  "detail": "错误描述信息"
}
```

3.3 响应头

头部字段	说明	示例
Content-Type	响应内容类型	application/json
Content-Length	响应内容长度	1234
Date	响应时间	Mon, 01 Jan 2024 12:00:00 GMT

4. 状态码说明

4.1 HTTP 状态码

状态码	说明	描述
200	OK	请求成功
201	Created	资源创建成功
204	No Content	请求成功但无返回内容
400	Bad Request	请求参数错误
401	Unauthorized	未授权（未来版本）
403	Forbidden	禁止访问（未来版本）
404	Not Found	资源不存在
500	Internal Server Error	服务器内部错误
503	Service Unavailable	服务不可用

4.2 业务状态

流水线状态（status 字段）：

状态值	说明
idle	空闲，未启动
running	运行中
completed	已完成
failed	执行失败
stopped	已停止

Agent 状态：

状态值	说明
idle	空闲
running	运行中
success	执行成功
error	执行错误

5. 文件上传接口

5.1 上传图片

上传参考图表图片文件。

基本信息

POST /api/upload

Content-Type: multipart/form-data

请求参数

参数名	类型	必填	说明
file	File	是	图片文件

参数约束

- 文件格式: PNG, JPG, JPEG
- 文件大小: 最大 10MB (10485760 字节)
- 文件内容: 必须是有效的图片文件

请求示例

```
curl -X POST "http://localhost:8001/api/upload" \
-H "Content-Type: multipart/form-data" \
-F "file=@/path/to/chart.png"
```

响应参数

字段名	类型	说明
path	string	文件存储路径

字段名	类型	说明
filename	string	文件名（UUID生成）
size	integer	文件大小（字节）

响应示例

```
{
  "path": "storage/uploads/a1b2c3d4-e5f6-7890-abcd-ef1234567890.png",
  "filename": "a1b2c3d4-e5f6-7890-abcd-ef1234567890.png",
  "size": 245678
}
```

错误响应

文件过大:

```
{
  "detail": "文件大小超过限制 (10.0MB)"
}
```

状态码: 400

文件格式错误:

```
{
  "detail": "不支持的文件类型"
}
```

状态码: 400

6. 流水线管理接口

6.1 启动流水线

启动图表复现流水线，开始多智能体协作处理。

基本信息

```
POST /api/pipeline/start
Content-Type: application/json
```

请求参数

参数名	类型	必填	默认值	说明
image_path	string	是	-	上传图片的路径
max_loops	integer	否	5	最大迭代轮数 (1-10)
threshold	float	否	0.75	验证通过阈值 (0-1)
model_provider	string	否	qwen	模型提供商

模型提供商选项

值	说明
qwen	阿里云通义千问
openai	OpenAI GPT
gemini	Google Gemini
doubao	字节豆包

请求示例

```
curl -X POST "http://localhost:8001/api/pipeline/start" \
-H "Content-Type: application/json" \
-d '{
  "image_path": "storage/uploads/a1b2c3d4-e5f6-7890-abcd-ef1234567890.png",
  "max_loops": 5,
  "threshold": 0.75,
  "model_provider": "qwen"
}'
```

响应参数

字段名	类型	说明
pipeline_id	string	流水线唯一标识符
status	string	流水线状态 (running)
message	string	提示信息

响应示例

```
{
  "pipeline_id": "pipeline_20240101_120000_abc123",
  "status": "running",
  "message": ""
}
```

```
"message": "流水线已启动"
}
```

错误响应

图片文件不存在:

```
{
  "detail": "图片文件不存在"
}
```

状态码: 404

参数验证失败:

```
{
  "detail": "max_loops 必须在 1-10 之间"
}
```

状态码: 400

6.2 查询流水线状态

查询指定流水线的执行状态和进度。

基本信息

```
GET /api/pipeline/{pipeline_id}/status
```

路径参数

参数名	类型	必填	说明
pipeline_id	string	是	流水线ID

请求示例

```
curl -X GET
"http://localhost:8001/api/pipeline/pipeline_20240101_120000_abc123/status"
```

响应参数

字段名	类型	说明
pipeline_id	string	流水线ID
status	string	流水线状态
current_round	integer	当前执行轮次
max_rounds	integer	最大轮次
agents	array	Agent 状态列表

Agent 状态对象

字段名	类型	说明
agent_id	string	Agent 标识
status	string	Agent 状态
task	string	当前任务描述
progress	integer	进度百分比 (0-100)
message	string	状态消息

响应示例

```
{
  "pipeline_id": "pipeline_20240101_120000_abc123",
  "status": "running",
  "current_round": 2,
  "max_rounds": 5,
  "agents": [
    {
      "agent_id": "agent1",
      "status": "success",
      "task": "代码生成",
      "progress": 100,
      "message": "代码生成完成"
    },
    {
      "agent_id": "agent2",
      "status": "running",
      "task": "视觉评估",
      "progress": 60,
      "message": "正在分析图表相似度..."
    },
    {
      "agent_id": "agent3",
      "status": "idle",
      "task": "代码分析",
      "progress": 0,

```



```
    "message": "等待执行"
  },
  {
    "agent_id": "agent4",
    "status": "idle",
    "task": "反馈修订",
    "progress": 0,
    "message": "等待执行"
  }
]
```

错误响应

流水线不存在:

```
{
  "detail": "流水线不存在"
}
```

状态码: 404

6.3 停止流水线

停止正在运行的流水线。

基本信息

```
POST /api/pipeline/{pipeline_id}/stop
```

路径参数

参数名	类型	必填	说明
pipeline_id	string	是	流水线ID

请求示例

```
curl -X POST
"http://localhost:8001/api/pipeline/pipeline_20240101_120000_abc123/stop"
```

响应参数

字段名	类型	说明
message	string	操作结果消息

响应示例

```
{
  "message": "流水线已停止"
}
```

错误响应

流水线不存在:

```
{
  "detail": "流水线不存在"
}
```

状态码: 404

流水线未在运行:

```
{
  "detail": "流水线未在运行"
}
```

状态码: 400

7. 结果查询接口

7.1 获取执行结果

获取流水线执行完成后的结果，包括生成的代码、图片、评分等。

基本信息

```
GET /api/results/{pipeline_id}
```

路径参数

参数名	类型	必填	说明
pipeline_id	string	是	流水线ID

查询参数

参数名	类型	必填	说明
round	integer	否	指定轮次（保留参数）

请求示例

```
curl -X GET "http://localhost:8001/api/results/pipeline_20240101_120000_abc123"
```

响应参数

字段名	类型	说明
pipeline_id	string	流水线ID
code	string	生成的 Matplotlib 代码
image_url	string	生成图片的URL
score	float	综合评分 (0-1)
passed	boolean	是否通过验证
dimensions	object	各维度评分
reports	array	详细报告列表

维度评分对象 (dimensions)

字段名	类型	说明
color	float	颜色一致性分数 (0-1)
text	float	文本一致性分数 (0-1)
structure	float	结构一致性分数 (0-1)
vlm	float	VLM 感知分数 (0-1)

报告对象 (reports)

字段名	类型	说明
title	string	报告标题
timestamp	string	生成时间 (ISO 8601)

字段名	类型	说明
content	string	报告内容

响应示例

```
{
  "pipeline_id": "pipeline_20240101_120000_abc123",
  "code": "import matplotlib.pyplot as plt\nimport numpy as np\n\n# 创建图表\nfig, ax = plt.subplots(figsize=(10, 6))\n\n# 数据\nx = np.array([1, 2, 3, 4, 5])\ny = np.array([2, 4, 6, 8, 10])\n\n# 绘制\nax.plot(x, y, marker='o', linewidth=2, color='#1f77b4')\nax.set_xlabel('X轴', fontsize=12)\nax.set_ylabel('Y轴', fontsize=12)\nax.set_title('示例图表', fontsize=14)\nax.grid(True, alpha=0.3)\n\nplt.tight_layout()\nplt.savefig('generated_chart.png', dpi=300, bbox_inches='tight')\nplt.show()",
  "image_url": "http://localhost:8001/outputs/pipeline_20240101_120000_abc123/generated_chart.png",
  "score": 0.8234,
  "passed": true,
  "dimensions": {
    "color": 0.8567,
    "text": 0.7823,
    "structure": 0.8456,
    "vlm": 0.8091
  },
  "reports": [
    {
      "title": "report agent2 round1",
      "timestamp": "2024-01-01T12:05:30",
      "content": "视觉评估报告：\n\n整体相似度：85%\n\n优点：\n- 颜色方案基本一致\n- 图表类型正确\n- 布局合理\n\n改进建议：\n- 调整坐标轴标签字体大小\n- 优化网格线透明度"
    },
    {
      "title": "report agent3 round1",
      "timestamp": "2024-01-01T12:06:15",
      "content": "代码分析报告：\n\n代码质量：良好\n\n优点：\n- 代码结构清晰\n- 注释完整\n- 参数设置合理\n\n建议：\n- 可以添加异常处理\n- 考虑参数化配置"
    }
  ]
}
```

错误响应

流水线不存在:

```
{
  "detail": "流水线不存在"
}
```

状态码: 404

流水线尚未完成:

```
{
  "detail": "流水线尚未完成"
}
```

状态码: 400

8. 文件下载接口

8.1 下载代码文件

下载生成的 Python 代码文件。

基本信息

```
GET /api/download/{pipeline_id}/code
```

路径参数

参数名	类型	必填	说明
pipeline_id	string	是	流水线ID

请求示例

```
curl -X GET
"http://localhost:8001/api/download/pipeline_20240101_120000_abc123/code" \
-o generated_chart.py
```

响应

返回文件流，Content-Type: text/x-python

响应头

头部字段	值
Content-Type	text/x-python
Content-Disposition	attachment; filename="generated_chart.py"

错误响应

流水线不存在:

```
{
  "detail": "流水线不存在"
}
```

状态码: 404

代码文件不存在:

```
{
  "detail": "代码文件不存在"
}
```

状态码: 404

8.2 下载图片文件

下载生成的图表图片。

基本信息

```
GET /api/download/{pipeline_id}/image
```

路径参数

参数名	类型	必填	说明
pipeline_id	string	是	流水线ID

请求示例

```
curl -X GET
"http://localhost:8001/api/download/pipeline_20240101_120000_abc123/image" \
-o generated_chart.png
```

响应

返回文件流，Content-Type: image/png

响应头

头部字段	值
Content-Type	image/png
Content-Disposition	attachment; filename="generated_chart.png"

错误响应

流水线不存在:

```
{
  "detail": "流水线不存在"
}
```

状态码: 404

图片文件不存在:

```
{
  "detail": "图片文件不存在"
}
```

状态码: 404

8.3 下载报告文件

下载验证报告文本文件。

基本信息

```
GET /api/download/{pipeline_id}/report
```

路径参数

参数名	类型	必填	说明
pipeline_id	string	是	流水线ID

请求示例

```
curl -X GET
"http://localhost:8001/api/download/pipeline_20240101_120000_abc123/report" \
-o validation_report.txt
```

响应

返回文件流，Content-Type: text/plain

响应头

头部字段	值
Content-Type	text/plain
Content-Disposition	attachment; filename="validation_report.txt"

错误响应

流水线不存在:

```
{
  "detail": "流水线不存在"
}
```

状态码: 404

报告文件不存在:

```
{
  "detail": "报告文件不存在"
}
```

状态码: 404

9. 历史记录接口

9.1 获取历史记录列表

获取所有已完成或失败的流水线历史记录。

基本信息


```
GET /api/history
```

请求示例

```
curl -X GET "http://localhost:8001/api/history"
```

响应参数

字段名	类型	说明
history	array	历史记录列表

历史记录对象

字段名	类型	说明
id	string	流水线ID
filename	string	原始文件名
timestamp	string	创建时间 (ISO 8601)
rounds	integer	执行轮次
score	float	最终得分 (0-1)
status	string	状态 (completed/failed/stopped)

响应示例

```
{
  "history": [
    {
      "id": "pipeline_20240101_150000_xyz789",
      "filename": "chart2.png",
      "timestamp": "2024-01-01T15:00:00",
      "rounds": 3,
      "score": 0.8567,
      "status": "completed"
    },
    {
      "id": "pipeline_20240101_120000_abc123",
      "filename": "chart1.png",
      "timestamp": "2024-01-01T12:00:00",
      "rounds": 5,
      "score": 0.8234,
      "status": "completed"
    }
  ]
}
```

```
    },
    {
      "id": "pipeline_20240101_100000_def456",
      "filename": "chart0.png",
      "timestamp": "2024-01-01T10:00:00",
      "rounds": 2,
      "score": 0.6543,
      "status": "failed"
    }
  ]
}
```

9.2 删除历史记录

删除指定的历史记录及其相关文件。

基本信息

```
DELETE /api/history/{pipeline_id}
```

路径参数

参数名	类型	必填	说明
pipeline_id	string	是	流水线ID

请求示例

```
curl -X DELETE "http://localhost:8001/api/history/pipeline_20240101_120000_abc123"
```

响应参数

字段名	类型	说明
message	string	操作结果消息

响应示例

```
{
  "message": "历史记录已删除"
}
```

错误响应

历史记录不存在:

```
{
  "detail": "历史记录不存在"
}
```

状态码: 404

删除失败:

```
{
  "detail": "删除失败：文件系统错误"
}
```

状态码: 500

10. WebSocket 接口

10.1 实时通信连接

建立 WebSocket 连接，实时接收流水线执行进度和状态更新。

基本信息

```
WebSocket /ws/{pipeline_id}
```

路径参数

参数名	类型	必填	说明
pipeline_id	string	是	流水线ID

连接示例

```
// JavaScript 示例
const ws = new
WebSocket('ws://localhost:8001/ws/pipeline_20240101_120000_abc123');

ws.onopen = () => {
  console.log('WebSocket 连接已建立');
};

ws.onmessage = (event) => {
  const message = JSON.parse(event.data);
```

```
    console.log('收到消息:', message);
  };

  ws.onerror = (error) => {
    console.error('WebSocket 错误:', error);
  };

  ws.onclose = () => {
    console.log('WebSocket 连接已关闭');
  };
}
```

消息格式

所有消息均为 JSON 格式：

```
{
  "type": "消息类型",
  "payload": {
    // 消息内容
  }
}
```

10.2 消息类型

10.2.1 Agent 状态更新

当 Agent 状态发生变化时推送。

消息类型: `agent_update`

Payload 结构:

字段名	类型	说明
agent_id	string	Agent 标识
status	string	Agent 状态
task	string	任务描述
progress	integer	进度 (0-100)
message	string	状态消息

示例:

```
{
  "type": "agent_update",
  "payload": {
    "agent_id": "agent1",
  }
}
```

```
    "status": "running",
    "task": "代码生成",
    "progress": 50,
    "message": "正在分析图表特征..."
  }
}
```

10.2.2 流水线状态更新

当流水线整体状态发生变化时推送。

消息类型: pipeline_update

Payload 结构:

字段名	类型	说明
pipeline_id	string	流水线ID
status	string	流水线状态
current_round	integer	当前轮次
message	string	状态消息

示例:

```
{
  "type": "pipeline_update",
  "payload": {
    "pipeline_id": "pipeline_20240101_120000_abc123",
    "status": "running",
    "current_round": 2,
    "message": "开始第2轮迭代"
  }
}
```

10.2.3 验证结果更新

当验证器完成评分时推送。

消息类型: validation_update

Payload 结构:

字段名	类型	说明
round	integer	轮次
score	float	综合得分

字段名	类型	说明
passed	boolean	是否通过
dimensions	object	各维度得分

示例:

```
{
  "type": "validation_update",
  "payload": {
    "round": 2,
    "score": 0.8234,
    "passed": true,
    "dimensions": {
      "color": 0.8567,
      "text": 0.7823,
      "structure": 0.8456,
      "vlm": 0.8091
    }
  }
}
```

10.2.4 流水线完成

当流水线执行完成时推送。

消息类型: pipeline_completed

Payload 结构:

字段名	类型	说明
pipeline_id	string	流水线ID
status	string	最终状态
total_rounds	integer	总轮次
final_score	float	最终得分
message	string	完成消息

示例:

```
{
  "type": "pipeline_completed",
  "payload": {
    "pipeline_id": "pipeline_20240101_120000_abc123",
    "status": "completed",
    "total_rounds": 3,
  }
}
```

```
    "final_score": 0.8234,
    "message": "流水线执行完成，验证通过"
  }
}
```

10.2.5 错误消息

当发生错误时推送。

消息类型: error

Payload 结构:

字段名	类型	说明
error_code	string	错误代码
error_message	string	错误描述
details	string	详细信息

示例:

```
{
  "type": "error",
  "payload": {
    "error_code": "MODEL_API_ERROR",
    "error_message": "模型API调用失败",
    "details": "Connection timeout after 30 seconds"
  }
}
```

10.3 心跳机制

WebSocket 连接建立后，服务器会定期发送心跳消息保持连接。

消息类型: heartbeat

示例:

```
{
  "type": "heartbeat",
  "payload": {
    "timestamp": "2024-01-01T12:05:30"
  }
}
```

客户端无需响应心跳消息，但可以通过心跳判断连接是否正常。

11. 错误处理

11.1 错误响应格式

所有错误响应均采用统一格式：

```
{
  "detail": "错误描述信息"
}
```

11.2 常见错误

11.2.1 参数验证错误

状态码: 400

示例:

```
{
  "detail": "max_loops 必须在 1-10 之间"
}
```

可能原因:

- 参数类型错误
- 参数值超出范围
- 必填参数缺失

11.2.2 资源不存在

状态码: 404

示例:

```
{
  "detail": "流水线不存在"
}
```

可能原因:

- 流水线ID错误
- 资源已被删除
- 路径参数错误

11.2.3 业务逻辑错误

状态码: 400

示例:

```
{
  "detail": "流水线未在运行"
}
```

可能原因:

- 操作不符合当前状态
- 业务规则限制

11.2.4 服务器内部错误

状态码: 500

示例:

```
{
  "detail": "启动失败：模型API调用超时"
}
```

可能原因:

- 模型API异常
- 文件系统错误
- 内部服务异常

11.3 错误处理建议

客户端处理

```
async function callAPI() {
  try {
    const response = await fetch('http://localhost:8001/api/pipeline/start', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({
        image_path: 'storage/uploads/test.png',
        max_loops: 5,
        threshold: 0.75
      })
    })
  } catch {
    // 处理错误
  }

  if (!response.ok) {
```

```
const error = await response.json();
console.error('API错误:', error.detail);

// 根据状态码处理
switch (response.status) {
  case 400:
    alert('请求参数错误: ' + error.detail);
    break;
  case 404:
    alert('资源不存在: ' + error.detail);
    break;
  case 500:
    alert('服务器错误: ' + error.detail);
    break;
  default:
    alert('未知错误: ' + error.detail);
}
return null;
}

const data = await response.json();
return data;
} catch (error) {
  console.error('网络错误:', error);
  alert('网络连接失败, 请检查网络设置');
  return null;
}
}
```

重试策略

对于临时性错误（如网络超时、服务暂时不可用），建议实现重试机制：

```
async function callAPIWithRetry(url, options, maxRetries = 3) {
  for (let i = 0; i < maxRetries; i++) {
    try {
      const response = await fetch(url, options);

      if (response.ok) {
        return await response.json();
      }

      // 5xx 错误可以重试
      if (response.status >= 500 && i < maxRetries - 1) {
        await sleep(1000 * Math.pow(2, i)); // 指数退避
        continue;
      }

      // 其他错误不重试
      const error = await response.json();
      throw new Error(error.detail);
    }
  }
}
```

```
    } catch (error) {
      if (i === maxRetries - 1) {
        throw error;
      }
      await sleep(1000 * Math.pow(2, i));
    }
  }
}

function sleep(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
```

12. 使用示例

12.1 完整流程示例

以下是一个完整的图表复现流程示例。

12.1.1 Python 示例

```
import requests
import time
import json

BASE_URL = "http://localhost:8001"

def upload_image(file_path):
    """上传图片"""
    with open(file_path, 'rb') as f:
        files = {'file': f}
        response = requests.post(f"{BASE_URL}/api/upload", files=files)
        response.raise_for_status()
        return response.json()

def start_pipeline(image_path, max_loops=5, threshold=0.75):
    """启动流水线"""
    data = {
        "image_path": image_path,
        "max_loops": max_loops,
        "threshold": threshold,
        "model_provider": "qwen"
    }
    response = requests.post(f"{BASE_URL}/api/pipeline/start", json=data)
    response.raise_for_status()
    return response.json()

def get_pipeline_status(pipeline_id):
    """查询流水线状态"""
    response = requests.get(f"{BASE_URL}/api/pipeline/{pipeline_id}/status")
```

```
response.raise_for_status()
return response.json()

def get_results(pipeline_id):
    """获取结果"""
    response = requests.get(f"{BASE_URL}/api/results/{pipeline_id}")
    response.raise_for_status()
    return response.json()

def download_code(pipeline_id, output_path):
    """下载代码"""
    response = requests.get(f"{BASE_URL}/api/download/{pipeline_id}/code")
    response.raise_for_status()
    with open(output_path, 'wb') as f:
        f.write(response.content)

# 主流程
def main():
    # 1. 上传图片
    print("上传图片...")
    upload_result = upload_image("chart.png")
    print(f"上传成功: {upload_result['filename']}")

    # 2. 启动流水线
    print("启动流水线...")
    start_result = start_pipeline(upload_result['path'])
    pipeline_id = start_result['pipeline_id']
    print(f"流水线ID: {pipeline_id}")

    # 3. 轮询状态
    print("等待执行完成...")
    while True:
        status = get_pipeline_status(pipeline_id)
        print(f"状态: {status['status']}, 轮次: {status['current_round']}/{status['max_rounds']}")

        if status['status'] in ['completed', 'failed', 'stopped']:
            break

        time.sleep(5)

    # 4. 获取结果
    if status['status'] == 'completed':
        print("获取结果...")
        results = get_results(pipeline_id)
        print(f"最终得分: {results['score']:.4f}")
        print(f"是否通过: {results['passed']}")
        print(f"各维度得分:")
        print(f"  颜色: {results['dimensions']['color']:.4f}")
        print(f"  文本: {results['dimensions']['text']:.4f}")
        print(f"  结构: {results['dimensions']['structure']:.4f}")
        print(f"  VLM: {results['dimensions']['vlm']:.4f}")

    # 5. 下载代码
```

```
        print("下载代码...")
        download_code(pipeline_id, "generated_chart.py")
        print("完成!")
    else:
        print(f"流水线执行失败: {status['status']}")

if __name__ == "__main__":
    main()
```

12.1.2 JavaScript 示例

```
const BASE_URL = 'http://localhost:8001';

// 上传图片
async function uploadImage(file) {
    const formData = new FormData();
    formData.append('file', file);

    const response = await fetch(`${BASE_URL}/api/upload`, {
        method: 'POST',
        body: formData
    });

    if (!response.ok) {
        throw new Error('上传失败');
    }

    return await response.json();
}

// 启动流水线
async function startPipeline(imagePath, maxLoops = 5, threshold = 0.75) {
    const response = await fetch(`${BASE_URL}/api/pipeline/start`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({
            image_path: imagePath,
            max_loops: maxLoops,
            threshold: threshold,
            model_provider: 'qwen'
        })
    });

    if (!response.ok) {
        throw new Error('启动失败');
    }

    return await response.json();
}
```

```
// 查询流水线状态
async function getPipelineStatus(pipelineId) {
  const response = await fetch(`${BASE_URL}/api/pipeline/${pipelineId}/status`);

  if (!response.ok) {
    throw new Error('查询失败');
  }

  return await response.json();
}

// 获取结果
async function getResults(pipelineId) {
  const response = await fetch(`${BASE_URL}/api/results/${pipelineId}`);

  if (!response.ok) {
    throw new Error('获取结果失败');
  }

  return await response.json();
}

// WebSocket 连接
function connectWebSocket(pipelineId, onMessage) {
  const ws = new WebSocket(`ws://localhost:8001/ws/${pipelineId}`);

  ws.onopen = () => {
    console.log('WebSocket 连接已建立');
  };

  ws.onmessage = (event) => {
    const message = JSON.parse(event.data);
    onMessage(message);
  };

  ws.onerror = (error) => {
    console.error('WebSocket 错误:', error);
  };

  ws.onclose = () => {
    console.log('WebSocket 连接已关闭');
  };

  return ws;
}

// 主流程
async function main() {
  try {
    // 1. 上传图片
    console.log('上传图片...');
    const fileInput = document.getElementById('fileInput');
    const file = fileInput.files[0];
```

```
const uploadResult = await uploadImage(file);
console.log('上传成功:', uploadResult.filename);

// 2. 启动流水线
console.log('启动流水线...');
const startResult = await startPipeline(uploadResult.path);
const pipelineId = startResult.pipeline_id;
console.log('流水线ID:', pipelineId);

// 3. 建立 WebSocket 连接接收实时更新
const ws = connectWebSocket(pipelineId, (message) => {
  console.log('收到消息:', message);

  switch (message.type) {
    case 'agent_update':
      console.log(`Agent ${message.payload.agent_id}:`);
      console.log(`${message.payload.message}`);
      break;
    case 'pipeline_update':
      console.log(`流水线状态: ${message.payload.status}, 轮次:`);
      console.log(`${message.payload.current_round}`);
      break;
    case 'validation_update':
      console.log(`验证得分: ${message.payload.score}`);
      break;
    case 'pipeline_completed':
      console.log('流水线执行完成!');
      ws.close();
      loadResults(pipelineId);
      break;
    case 'error':
      console.error('错误:', message.payload.error_message);
      ws.close();
      break;
  }
});

} catch (error) {
  console.error('错误:', error);
}

}

// 加载结果
async function loadResults(pipelineId) {
  try {
    const results = await getResults(pipelineId);
    console.log('最终得分:', results.score);
    console.log('是否通过:', results.passed);
    console.log('各维度得分:', results.dimensions);

    // 显示结果
    document.getElementById('score').textContent = results.score.toFixed(4);
    document.getElementById('generatedImage').src = results.image_url;
    document.getElementById('code').textContent = results.code;
  }
}
```

```
    } catch (error) {  
      console.error('加载结果失败:', error);  
    }  
  }  
}
```

12.1.3 cURL 示例

```
#!/bin/bash  
  
BASE_URL="http://localhost:8001"  
  
# 1. 上传图片  
echo "上传图片..."  
UPLOAD_RESPONSE=$(curl -s -X POST "$BASE_URL/api/upload" \  
  -F "file=@chart.png")  
  
IMAGE_PATH=$(echo $UPLOAD_RESPONSE | jq -r '.path')  
echo "上传成功: $IMAGE_PATH"  
  
# 2. 启动流水线  
echo "启动流水线..."  
START_RESPONSE=$(curl -s -X POST "$BASE_URL/api/pipeline/start" \  
  -H "Content-Type: application/json" \  
  -d "{  
    \"image_path\": \"$IMAGE_PATH\",  
    \"max_loops\": 5,  
    \"threshold\": 0.75,  
    \"model_provider\": \"qwen\"  
  }")  
  
PIPELINE_ID=$(echo $START_RESPONSE | jq -r '.pipeline_id')  
echo "流水线ID: $PIPELINE_ID"  
  
# 3. 轮询状态  
echo "等待执行完成..."  
while true; do  
  STATUS_RESPONSE=$(curl -s -X GET "$BASE_URL/api/pipeline/$PIPELINE_ID/status")  
  STATUS=$(echo $STATUS_RESPONSE | jq -r '.status')  
  ROUND=$(echo $STATUS_RESPONSE | jq -r '.current_round')  
  MAX_ROUNDS=$(echo $STATUS_RESPONSE | jq -r '.max_rounds')  
  
  echo "状态: $STATUS, 轮次: $ROUND/$MAX_ROUNDS"  
  
  if [ "$STATUS" = "completed" ] || [ "$STATUS" = "failed" ] || [ "$STATUS" =  
"stopped" ]; then  
    break  
  fi  
  
  sleep 5  
done
```



```
# 4. 获取结果
if [ "$STATUS" = "completed" ]; then
    echo "获取结果..."
    RESULTS=$(curl -s -X GET "$BASE_URL/api/results/$PIPELINE_ID")

    SCORE=$(echo $RESULTS | jq -r '.score')
    PASSED=$(echo $RESULTS | jq -r '.passed')

    echo "最终得分: $SCORE"
    echo "是否通过: $PASSED"

# 5. 下载代码
echo "下载代码..."
curl -s -X GET "$BASE_URL/api/download/$PIPELINE_ID/code" \
    -o "generated_chart.py"

    echo "完成!"
else
    echo "流水线执行失败: $STATUS"
fi
```

12.2 批量处理示例

批量处理多个图表文件。

```
import os
import requests
import time
from concurrent.futures import ThreadPoolExecutor, as_completed

BASE_URL = "http://localhost:8001"

def process_single_chart(file_path):
    """处理单个图表"""
    try:
        # 上传
        with open(file_path, 'rb') as f:
            files = {'file': f}
            upload_resp = requests.post(f"{BASE_URL}/api/upload", files=files)
            upload_resp.raise_for_status()
            upload_data = upload_resp.json()

        # 启动流水线
        start_data = {
            "image_path": upload_data['path'],
            "max_loops": 5,
            "threshold": 0.75
        }
        start_resp = requests.post(f"{BASE_URL}/api/pipeline/start",
            json=start_data)
        start_resp.raise_for_status()
```

```
pipeline_id = start_resp.json()['pipeline_id']

# 等待完成
while True:
    status_resp = requests.get(f"{BASE_URL}/api/pipeline/{pipeline_id}/status")
    status_resp.raise_for_status()
    status_data = status_resp.json()

    if status_data['status'] in ['completed', 'failed', 'stopped']:
        break

    time.sleep(5)

# 获取结果
if status_data['status'] == 'completed':
    results_resp = requests.get(f"{BASE_URL}/api/results/{pipeline_id}")
    results_resp.raise_for_status()
    results = results_resp.json()

    return {
        'file': file_path,
        'pipeline_id': pipeline_id,
        'score': results['score'],
        'passed': results['passed'],
        'status': 'success'
    }
else:
    return {
        'file': file_path,
        'pipeline_id': pipeline_id,
        'status': 'failed'
    }

except Exception as e:
    return {
        'file': file_path,
        'status': 'error',
        'error': str(e)
    }

def batch_process(input_dir, max_workers=3):
    """批量处理"""
    # 获取所有图片文件
    files = []
    for filename in os.listdir(input_dir):
        if filename.lower().endswith(('.png', '.jpg', '.jpeg')):
            files.append(os.path.join(input_dir, filename))

    print(f"找到 {len(files)} 个图片文件")

    # 并发处理
    results = []
    with ThreadPoolExecutor(max_workers=max_workers) as executor:
```

```

futures = {executor.submit(process_single_chart, f): f for f in files}

for future in as_completed(futures):
    result = future.result()
    results.append(result)

    if result['status'] == 'success':
        print(f"✓ {result['file']}: 得分 {result['score']:.4f}")
    else:
        print(f"X {result['file']}: {result['status']}")

# 统计
success_count = sum(1 for r in results if r['status'] == 'success')
print(f"\n完成: {success_count}/{len(files)} 成功")

return results

if __name__ == "__main__":
    results = batch_process("input_charts", max_workers=3)

```

12.3 错误处理示例

完善的错误处理示例。

```

import requests
import time
from requests.exceptions import RequestException, Timeout, ConnectionError

class APIClient:
    def __init__(self, base_url, max_retries=3, timeout=30):
        self.base_url = base_url
        self.max_retries = max_retries
        self.timeout = timeout

    def _request_with_retry(self, method, url, **kwargs):
        """带重试的请求"""
        for attempt in range(self.max_retries):
            try:
                response = requests.request(
                    method,
                    url,
                    timeout=self.timeout,
                    **kwargs
                )
                response.raise_for_status()
                return response
            except Timeout:
                if attempt < self.max_retries - 1:
                    wait_time = 2 ** attempt
                    print(f"请求超时, {wait_time}秒后重试...")

```

```
        time.sleep(wait_time)
    else:
        raise Exception("请求超时，已达最大重试次数")

except ConnectionError:
    if attempt < self.max_retries - 1:
        wait_time = 2 ** attempt
        print(f"连接失败，{wait_time}秒后重试...")
        time.sleep(wait_time)
    else:
        raise Exception("连接失败，请检查网络或服务器状态")

except requests.HTTPError as e:
    # 4xx 错误不重试
    if 400 <= e.response.status_code < 500:
        error_detail = e.response.json().get('detail', str(e))
        raise Exception(f"请求错误: {error_detail}")

    # 5xx 错误重试
    if attempt < self.max_retries - 1:
        wait_time = 2 ** attempt
        print(f"服务器错误，{wait_time}秒后重试...")
        time.sleep(wait_time)
    else:
        raise Exception("服务器错误，已达最大重试次数")

def upload_image(self, file_path):
    """上传图片"""
    try:
        with open(file_path, 'rb') as f:
            files = {'file': f}
            response = self._request_with_retry(
                'POST',
                f"{self.base_url}/api/upload",
                files=files
            )
            return response.json()
    except FileNotFoundError:
        raise Exception(f"文件不存在: {file_path}")
    except Exception as e:
        raise Exception(f"上传失败: {str(e)}")

def start_pipeline(self, image_path, max_loops=5, threshold=0.75):
    """启动流水线"""
    try:
        data = {
            "image_path": image_path,
            "max_loops": max_loops,
            "threshold": threshold
        }
        response = self._request_with_retry(
            'POST',
            f"{self.base_url}/api/pipeline/start",
            json=data
        )
```

```
    )
    return response.json()
except Exception as e:
    raise Exception(f"启动流水线失败: {str(e)}")

# 使用示例
try:
    client = APIClient("http://localhost:8001")

    # 上传图片
    upload_result = client.upload_image("chart.png")
    print(f"上传成功: {upload_result['filename']}")

    # 启动流水线
    start_result = client.start_pipeline(upload_result['path'])
    print(f"流水线已启动: {start_result['pipeline_id']}")

except Exception as e:
    print(f"错误: {str(e)}")
```

附录

A. 数据模型定义

UploadResponse

```
{
  "path": str,          # 文件存储路径
  "filename": str,      # 文件名
  "size": int           # 文件大小（字节）
}
```

PipelineConfig

```
{
  "image_path": str,    # 图片路径（必填）
  "max_loops": int,     # 最大轮数 1-10（默认5）
  "threshold": float,   # 阈值 0-1（默认0.75）
  "model_provider": str # 模型提供商（默认qwen）
}
```

PipelineStatus

```
{
  "pipeline_id": str,   # 流水线ID
}
```

```
"status": str,           # 状态
"current_round": int,     # 当前轮次
"max_rounds": int,       # 最大轮次
"agents": [AgentStatus]  # Agent状态列表
}
```

ResultsResponse

```
{
  "pipeline_id": str,      # 流水线ID
  "code": str,             # 生成的代码
  "image_url": str,        # 图片URL
  "score": float,          # 综合得分
  "passed": bool,          # 是否通过
  "dimensions": {          # 各维度得分
    "color": float,
    "text": float,
    "structure": float,
    "vlm": float
  },
  "reports": [Report]      # 报告列表
}
```

B. 环境变量配置

变量名	说明	示例
MODEL_PROVIDER	模型提供商	qwen
QWEN_API_KEY	通义千问密钥	sk-xxx
OPENAI_API_KEY	OpenAI密钥	sk-xxx
GEMINI_API_KEY	Gemini密钥	xxx
DOUBAO_API_KEY	豆包密钥	xxx
SERVER_HOST	服务器地址	localhost
SERVER_PORT	服务器端口	8001
TESSERACT_CMD	Tesseract路径	/usr/bin/tesseract